PLACEMENT INTERVIEW — MASTER REVISION

Python Data Analysis Master Revision Guide

NumPy · Pandas · Matplotlib · Seaborn — four guides compressed into one.

PART 1 — NumPy
arrays, indexing, operations, reshape, broadcasting

PART 2 — Pandas
load, select, filter, groupby, merge, pivot, clean

PART 3 — Matplotlib
figure/axes, all charts, subplots, styling

PART 4 — Seaborn
fig vs axes level, all statistical plots

HOW THIS DOC IS WRITTEN

Every function is given in one line: **what it is used for** + **how to write the code**. No long outputs, no extra story. Concepts that interviewers dig into (broadcasting, view vs copy, groupby, merge types, figure vs axes level) get a short box with a diagram.

Day before interview: read the mindmaps, the theory boxes in 0.3, all green *"In interview, say this"* boxes, then Part 5 (88 Q&A) and Part 6 (cheat sheet + 20 golden rules).

Covers every topic of the NumPy, Pandas, Matplotlib and Seaborn revision guides — nothing important dropped.

Contents

Part 0 — Big Picture

0.1 Master mindmap of the 4 libraries
0.2 How the 4 libraries fit together
0.3 Theory words + basic statistics

Part 1 — NumPy

1.1 Why NumPy + mindmap
1.2 Creating arrays
1.3 Attributes & dtype
1.4 List vs Array
1.5 Indexing, slicing, iteration
1.6 View vs Copy
1.7 Operations + axis
1.8 Reshaping
1.9 Fancy & Boolean indexing, np.where
1.10 Broadcasting
1.11 Important functions, random, NaN, linalg

Part 2 — Pandas

2.1 Series, DataFrame + mindmap
2.2 read_csv & saving
2.3 Inspect: head, info, describe
2.4 loc vs iloc
2.5 Filtering, isin, query
2.6 value_counts, sort_values, drop_duplicates
2.7 Series ops, str & dt
2.8 groupby + agg + transform
2.9 merge, join, concat
2.10 pivot_table, crosstab, melt
2.11 corr()
2.12 rename, set_index, reset_index
2.13 Missing values: dropna, fillna
2.14 dtypes & cleaning
2.15 apply, map, cut, qcut, assign, explode
2.16 Rolling, time series, copy rules
2.17 Pandas vs SQL + performance
2.18 df.plot()

Part 3 — Matplotlib

3.1 Architecture, Figure/Axes/Axis
3.2 pyplot vs Object-Oriented
3.3 All chart types + when to use
3.4 Subplots & GridSpec
3.5 Styling, colormaps, annotation, axis
3.6 3D, imshow, saving, animation

Part 4 — Seaborn

4.1 Figure-level vs Axes-level + map
4.2 Long vs wide data
4.3 All plots, family by family
4.4 heatmap, clustermap, grids
4.5 Styling & palettes

Part 5 & 6

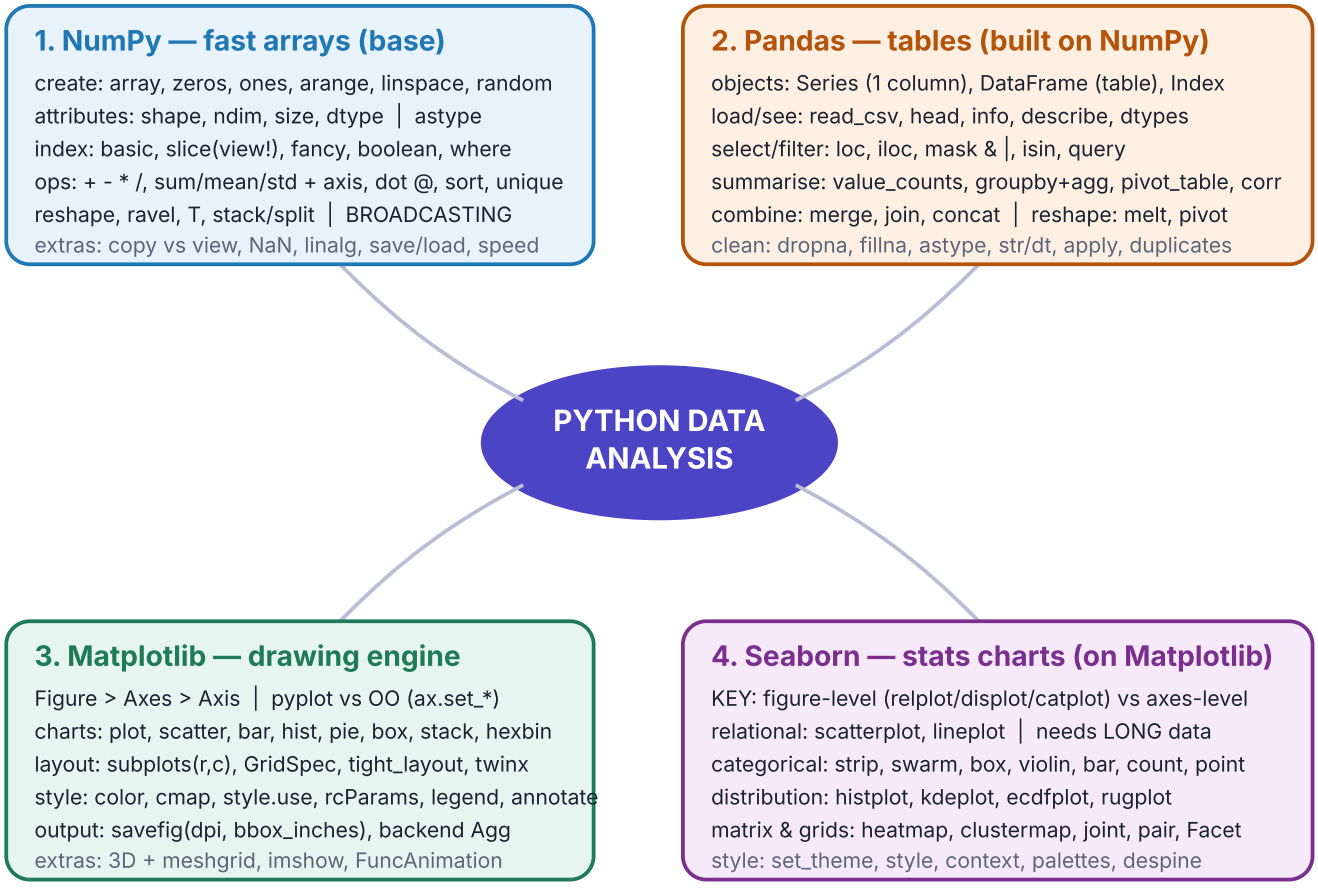
5 Rapid-fire interview Q&A (1–45)
5B More rapid-fire Q&A (46–88)
6 "I want to..." cheat sheet + 20 golden rules + mistakes

PART 0

The Big Picture

Before the details — see the whole map once, so every topic has a place to sit in your head.

0.1 Master Mindmap



Mindmap 0.1 — The whole syllabus on one page. Each box = one Part of this doc.

0.2 How the 4 Libraries Fit Together

Pandas is built on NumPy · Seaborn is built on Matplotlib · sklearn takes NumPy arrays



Real project flow: read data → clean → explore with charts → feed the model

IF THE INTERVIEWER ASKS "EXPLAIN YOUR DATA ANALYSIS PROCESS"

"First I load the data with `pd.read_csv` and look at `shape`, `info()`, `describe()` and `isnull().sum()`. Then I clean it — fix data types, handle missing values and duplicates. Then EDA: univariate with histplot and countplot, bivariate with scatterplot, boxplot and a correlation heatmap. NumPy is behind all of it for fast calculation, and for the model I pass the final numeric arrays to scikit-learn."

Library	Main object	One-line purpose	Import
NumPy	`ndarray`	Fast maths on same-type numbers, no Python loops (vectorization)	`import numpy as np`
Pandas	`Series`, `DataFrame`	Labelled tables: load, clean, filter, group, join	`import pandas as pd`
Matplotlib	`Figure`, `Axes`	Draw any chart with full control	`import matplotlib.pyplot as plt`
Seaborn	returns `Axes` / Grid	Statistical charts in one line from a DataFrame	`import seaborn as sns`

0.3 Theory Words You Must Know First

These words come inside almost every answer. If these are clear, the rest of the doc becomes easy.

▸ Types of analysis

Word	Meaning in simple words	Charts used
EDA	Exploratory Data Analysis — looking at the data first (shape, missing values, patterns) before building anything	all of them
Univariate	Studying **one** column alone	histogram, countplot, boxplot
Bivariate	Studying **two** columns together	scatter, boxplot by category, heatmap
Multivariate	**Three or more** columns together	hue, size, facet, pairplot

▸ Types of data

Type	Meaning	Example
Numerical	Numbers you can do maths on	age, salary, runs
• Continuous	Any value in a range, decimals allowed	height 5.4 ft
• Discrete	Countable whole numbers	number of children
Categorical	Labels / groups	city, gender
• Nominal	No order	Delhi, Pune, Goa
• Ordinal	Has an order	Low < Medium < High

Why this matters: the chart and the statistic you choose depend only on these two things — how many columns, and which type they are.

▸ Basic statistics (asked along with charts)

Term	Simple meaning	When to use / what it tells
Mean	Average = total ÷ count	Good only when there are no big outliers
Median	The middle value after sorting	**Safe with outliers** — that is why we fill salary with median
Mode	The most repeated value	For text / category columns
Variance	Average of squared distances from the mean	Measures spread (unit is squared)
Std deviation	Square root of variance	Spread in the **same unit** as the data
Range	max − min	Very rough spread
Percentile	Value below which that % of data lies	90th percentile salary = 90% earn less
Quartiles	Q1 = 25%, Q2 = median, Q3 = 75%	Used by the box plot
IQR	Q3 − Q1 — the middle 50% of the data	Outlier rule: below Q1−1.5×IQR or above Q3+1.5×IQR
Outlier	A value far away from the rest	Can be a data error or a genuine rare case — check before deleting
Skewness	Is the distribution tilted to one side?	Right skew (income): mean > median. Left skew: mean < median
Normal distribution	The bell curve, symmetric around the mean	68% of data within 1 std, 95% within 2 std
Bimodal	Distribution with **two peaks**	Means two different groups are mixed in one column
Correlation	How two numeric columns move together (−1 to +1)	Never proves cause

THE ONE LINE THAT EXPLAINS MEAN VS MEDIAN

"If 9 people earn 20,000 and 1 person earns 10 lakh, the **mean** becomes about 1.18 lakh, which describes nobody. The **median** stays 20,000, which is the real picture. So with outliers I always report and fill with median."

PART 1

NumPy

The base of everything. ndarray = fast, fixed-type, multi-dimensional array + vectorized maths.

1.1 Why NumPy + Mindmap

Point	Simple reason
Fast	Loops run in compiled C, not Python. This is called **vectorization** (10–100x faster).
Less memory	Raw numbers of one type stored side by side. A list stores pointers to separate objects.
Easy maths	`a + b`, `a * 2`, `a > 5` work on the whole array — no for loop.
Base library	Pandas, Matplotlib, scikit-learn, TensorFlow all use NumPy arrays.

ONE-LINE ANSWER: "WHAT IS NUMPY?"

"NumPy gives us the **ndarray** — a fast, fixed-type, n-dimensional array — and vectorized functions that work on the whole array at once without Python loops, because the real work happens in C."

1.2 Creating Arrays

An array is a **fixed block of memory holding items of one type**. So at creation time two things get decided: the **shape** and the **dtype**. These are the common ways to make one.

Function	What it is used for	Code
`np.array()`	Make an array from a list / list of lists	`np.array([[1,2],[3,4]])`
`np.zeros() / ones()`	Fill with 0 or 1 — placeholder before filling values	`np.zeros((2,3))`
`np.full()`	Fill with any one value	`np.full((2,2), 7)`
`np.eye() / identity()`	Identity matrix (1 on diagonal) — linear algebra	`np.eye(3)`
`np.arange()`	Range of numbers with a **step** (end not included)	`np.arange(0,10,2)`
`np.linspace()`	**Count** of evenly spaced numbers (end included) — for plotting x-values	`np.linspace(0,1,5)`
`np.empty()`	Fast, uninitialised (garbage values) — only when you will overwrite	`np.empty((2,2))`
`np.random.rand()`	Random floats 0–1, uniform	`np.random.rand(2,3)`
`np.random.randn()`	Random from normal distribution (mean 0, sd 1)	`np.random.randn(3)`
`np.random.randint()`	Random whole numbers in a range	`np.random.randint(1,7,10)`
`np.random.seed()`	Fix randomness so results repeat (needed in ML)	`np.random.seed(42)`

ARANGE VS LINSPACE — ASKED OFTEN

`arange(0,1,0.2)` → you give the **step**, end is excluded, and float steps can give wrong counts.
`linspace(0,1,5)` → you give **how many numbers** you want, end is included. For plotting always use linspace.

1.3 Attributes & dtype

Attributes are not functions — they are facts already stored about the array, so there are no brackets (`a.shape`, not `a.shape()`).

Attribute	Meaning	Example
`a.shape`	Rows, columns (tuple) — most used	`(2, 3)`
`a.ndim`	Number of dimensions (1-D, 2-D, 3-D)	`2`
`a.size`	Total number of items (rows × cols)	`6`
`a.dtype`	Data type of items (all same)	`int64`
`a.itemsize / nbytes`	Bytes per item / total bytes — memory questions	`8 / 48`
`a.T`	Transpose (rows become columns)	`a.T`
`a.astype()`	Change the data type	`a.astype(float)`

dtype basics: `int8/16/32/64`, `float32/64`, `bool`, `complex`, `str_`, `object`. An array has **one** dtype — mixing int and string makes everything a string. Using `float32` instead of `float64` halves memory, a good answer for "how do you reduce memory?"

1.4 Python List vs NumPy Array

Point	Python list	NumPy array
Data type	Mixed types allowed	One fixed type
Speed	Slow (Python loop)	Fast (C loop, vectorized)
Memory	More (pointers to objects)	Less (raw numbers together)
`a + b`	**Joins** the two lists	**Adds** item by item
`a * 2`	Repeats the list twice	Multiplies every item by 2
Size	Can grow / shrink easily	Fixed size (append makes a new array)
Maths tools	Almost none	mean, std, dot, linalg, random...

1.5 Indexing, Slicing, Iteration

Indexing = picking one item by its position. **Slicing** = picking a range. In 2-D you give two things separated by a comma: `[rows, columns]`.

```
a[2]            # 1-D: 3rd item (index starts at 0)
a[-1]           # last item
m[1, 2]         # 2-D: row 1, column 2   (better than m[1][2])
m[1]            # full row 1
m[:, 1]         # full column 1  <-- very common
m[0:2, 1:3]     # rows 0-1, columns 1-2   (end NOT included)
m[::2, ::-1]    # every 2nd row, columns reversed
t[0, 1, 2]      # 3-D: block 0, row 1, col 2

for row in m:        # loops row by row
for x in m.flat:     # loops every single item
np.nditer(m)         # efficient iterator
```

REMEMBER

Slice format is `start : stop : step` and **stop is never included**. In 2-D always use `m[row, col]` — `m[row][col]` creates an extra temporary array and is slower.

1.6 View vs Copy (favourite interview trap)

b = a[1:4] → VIEW (same memory)



b[0] = 99 also changed a[1]

Slicing, reshape, ravel(), .T → **view** (no new memory)
Fancy index, boolean index, .copy(), flatten() → **copy**

c = a[1:4].copy() → COPY



new memory — changing c does not touch a

Check with: b.base is a
(True = view, None = copy)

IN INTERVIEW, SAY THIS

"A NumPy slice returns a **view**, not a copy — it shares the same memory, so changing the slice changes the original. This is different from a Python list, where slicing copies. If I need an independent array I write `.copy()`. Fancy indexing and boolean indexing always return copies."

1.7 Operations & the axis Idea

THEORY: WHAT IS VECTORIZATION?

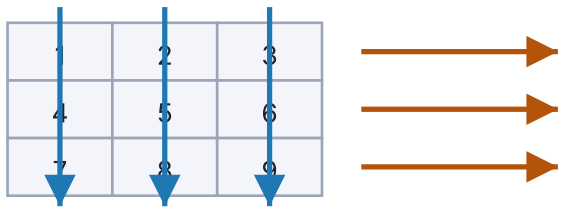
In normal Python you write a `for` loop and Python checks the type of every value, one by one — that is slow. In NumPy you write the operation on the **whole array** (`a + b`), and NumPy runs that loop inside compiled **C** code where all items are already the same type. The loop still happens, but it happens in fast machine code instead of slow Python. That is vectorization, and that is the real reason NumPy is 10–100 times faster.

ufunc (universal function) is the name for a function like `np.sqrt` that works item by item on the whole array in this vectorized way.

Arithmetic & universal functions (ufuncs) — item by item

```
a + b, a - b, a * b, a / b, a ** 2, a % 2    # element-wise
np.sqrt(a), np.exp(a), np.log(a), np.sin(a), np.round(a, 2), np.abs(a)
a > 5, (a > 2) & (a < 8)                      # gives a boolean array
```

axis=0 → goes DOWN the rows



result = one value per COLUMN [12 15 18]

axis=1 → goes ACROSS the columns

result = one value per ROW
[6 15 24]

Figure 1.1 — **axis = the direction that disappears.** Same rule works in Pandas.

Function	What it is used for	Code
`sum / mean / median`	Total, average, middle value	`a.sum(axis=0)`
`std / var`	Spread of data	`a.std()`
`min / max`	Smallest / biggest value	`a.max(axis=1)`
`argmin / argmax`	**Position** of smallest / biggest — not the value	`a.argmax()`
`cumsum / cumprod`	Running total / running product	`a.cumsum()`
`np.sort / argsort`	Sorted values / the indexes that would sort it	`np.sort(a)`
`np.unique`	Distinct values (+ their counts)	`np.unique(a, return_counts=True)`
`np.dot / @`	Matrix multiplication (ML, linear algebra)	`A @ B`
`np.concatenate`	Join arrays along an axis	`np.concatenate([a,b], axis=0)`
`vstack / hstack`	Stack vertically / horizontally (easier to remember)	`np.vstack([a,b])`
`np.split / hsplit`	Break one array into parts	`np.split(a, 3)`
`intersect1d / union1d`	Common values / all values (set operations)	`np.intersect1d(a,b)`
`setdiff1d / in1d`	Values only in a / membership check	`np.setdiff1d(a,b)`

TWO TRAPS

1. `a * b` is **element-wise** multiplication; `a @ b` is **matrix** multiplication. Interviewers love this.
2. `argmax` gives the **index**, `max` gives the **value**.

1.8 Reshaping

Theory point that impresses: in memory an array is stored as **one long line of numbers** (row by row, called C order). `reshape` only changes **how NumPy reads** that line — it does not move any data. That is why reshaping is almost free and usually returns a view.

Function	What it is used for	Code
`reshape()`	Change shape. Total items must match. `-1` = "you calculate it"	`a.reshape(2,-1)`
`ravel()`	Make it 1-D — returns a **view** (fast)	`a.ravel()`
`flatten()`	Make it 1-D — returns a **copy** (safe)	`a.flatten()`
`a.T / transpose()`	Swap rows and columns	`a.T`
`np.newaxis / expand_dims`	Add a dimension — needed for broadcasting and sklearn	`a[:, np.newaxis]`
`squeeze()`	Remove dimensions of size 1	`a.squeeze()`
`resize()`	Change shape even if size differs (repeats/pads)	`np.resize(a,(3,4))`

reshape vs resize: reshape needs the same number of items and returns a new view; resize can change the number of items. **ravel vs flatten:** view vs copy — this exact question comes in interviews.

1.9 Fancy Indexing, Boolean Indexing, np.where

Normal indexing takes positions one by one. **Fancy** indexing takes a whole **list of positions** at once. **Boolean** indexing first builds a True/False array (a **mask**) from a condition, and then keeps only the items where it is True. Boolean indexing is the base of filtering in Pandas too.

```
# FANCY: pick by a list of positions, in any order, repeats allowed
a[[0, 3, 1]]
m[[0, 2], :]                # rows 0 and 2
m[[0,1], [2,0]]             # items (0,2) and (1,0)

# BOOLEAN: pick by a condition (mask)
a[a > 50]
a[(a > 20) & (a < 80)]      # use & | ~ , NOT and/or/not
a[a % 2 == 0]
a[a > 50] = 0               # change values that match

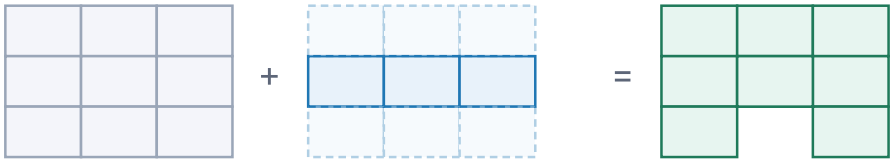
# np.where = if-else on the whole array
np.where(a > 50, 'pass', 'fail')
np.where(a % 2 != 0, -1, a)  # replace odd numbers with -1
np.where(a > 50)             # with 1 argument: gives the positions
np.count_nonzero(a > 50)     # how many match
(a > 50).sum()               # same thing (True = 1)
np.any(a > 90), np.all(a > 0)
```

Type	You give	Returns	Use for
Basic / slice	Ranges	**View**	Taking a block
Fancy	List of positions	**Copy**	Picking specific rows in your own order
Boolean	Condition	**Copy**	Filtering by a rule

1.10 Broadcasting

Broadcasting = NumPy automatically **stretches** a smaller array so two different shapes can work together, without actually copying data in memory.

(3,3) matrix + (3,) row → works



row is "copied" to every row

The 2 rules

Compare shapes from the RIGHT.
Each pair must be either
(a) equal, or (b) one of them = 1.

Otherwise → ValueError
(3,4) + (4,) OK · (3,4)+(3,) FAIL

```
a = np.array([[1,2,3],[4,5,6]])     # (2,3)
a + 10                              # scalar  -> works
a + np.array([10,20,30])            # (3,)    -> works (stretched down)
a + np.array([[10],[20]])           # (2,1)   -> works (stretched across)
a + np.array([10,20])               # (2,)    -> ValueError
# fix: make it a column
a + np.array([10,20])[:, np.newaxis]
```

WHY IT IS USEFUL (SAY THIS)

"Broadcasting lets me write `(data - data.mean(axis=0)) / data.std(axis=0)` to normalize every column in one line, with **no loop and no extra memory**, because NumPy only pretends to repeat the smaller array instead of really copying it."

1.11 Important Functions, Random, NaN, Linear Algebra

random module

`seed(42)`	repeat same random
`rand(2,3)`	uniform 0–1
`randn(5)`	normal (mean 0, sd 1)
`randint(1,7,10)`	whole numbers
`uniform(5,10,3)`	floats in a range
`choice(a, 3)`	random pick (sampling)
`shuffle(a)`	shuffle in place
`permutation(a)`	shuffled copy

NaN (missing values)

`np.nan`	the missing marker (a float)
`np.isnan(a)`	find NaNs (`== np.nan` never works)
`np.nanmean(a)`	mean ignoring NaN
`np.nan_to_num(a)`	replace NaN with 0

Other important functions

`np.percentile(a,[25,50,75])`	quartiles / IQR
`np.clip(a, 0, 100)`	force values into a range
`np.round / floor / ceil`	rounding
`np.cumsum(a)`	running total
`np.histogram(a, bins=5)`	counts per bin
`np.corrcoef(x, y)`	correlation
`np.save / np.load`	.npy binary file
`np.savetxt / loadtxt`	csv text file

Linear algebra (np.linalg)

`A @ B`	matrix multiply
`np.linalg.inv(A)`	inverse
`np.linalg.det(A)`	determinant
`np.linalg.solve(A,b)`	solve Ax = b
`np.linalg.eig(A)`	eigen values (PCA)
`np.linalg.norm(v)`	length / distance

NUMPY COMMON MISTAKES

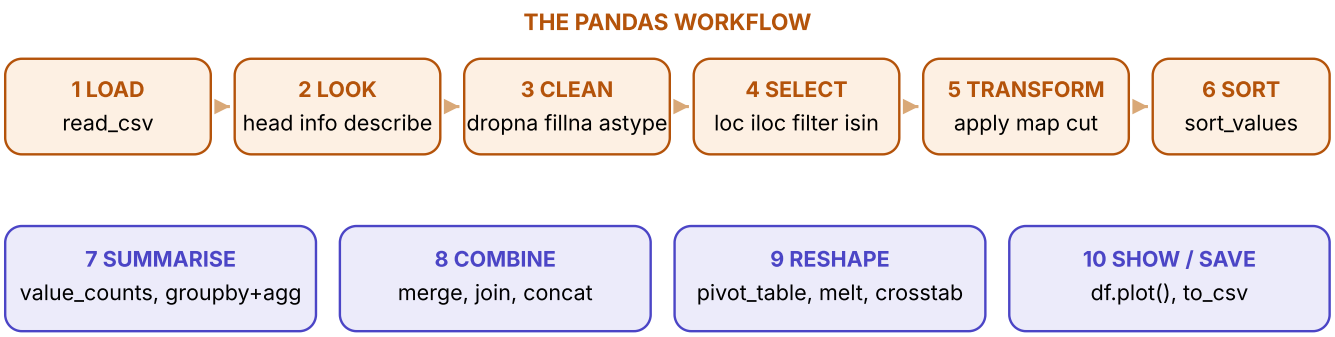
• Forgetting slices are views → original changes by mistake. • Using `and/or` instead of `&/|` in conditions. • Forgetting brackets: `(a>2)&(a<8)`. • `reshape` with a wrong total size. • Using `==` to check NaN. • Writing a Python loop where a vectorized operation exists.

PART 2

Pandas

Tables with labels. The library you will actually be tested on in a data analyst interview.

2.1 Series, DataFrame + Workflow Mindmap



This order is also the answer to "how do you approach a dataset?"

Object	Meaning	Code
Series	One column: values + index labels (1-D)	`pd.Series([10,20], index=['a','b'])`
DataFrame	Full table: many Series sharing one index (2-D)	`pd.DataFrame({'a':[1,2],'b':[3,4]})`
Index	The row labels. Can be numbers, text or dates	`df.index`, `df.columns`

THEORY: WHAT IS THE INDEX AND WHY DOES IT EXIST?

The **index** is the name tag of every row. NumPy rows have only positions (0,1,2); Pandas rows have **labels**. Because of the index Pandas can do three things NumPy cannot: look up a row by its name (`df.loc['R102']`), **align** two tables automatically when you add or merge them (same labels get matched, rest become NaN), and slice by date (`df.loc['2021']`). Alignment is the main idea — Pandas matches on labels, not on position.

Why Pandas and not NumPy? A NumPy array holds one data type and has no labels. A DataFrame holds numbers, text and dates in different columns, has column names and row labels, and has ready tools for missing data, grouping and joining. (Pandas = "Panel Data", built on NumPy.)

```
df['city']            # one column  -> Series
df[['city','age']]    # many columns -> DataFrame (double brackets!)
df['new'] = df['a'] * 2        # add a column
df.drop(columns=['a'], inplace=True)
```

2.2 read_csv() & Saving

A CSV file is just plain text with commas. `read_csv` reads it and **guesses** the type of every column. That guess is often wrong (dates read as text, pin codes read as numbers), so these parameters exist to control it.

Parameter	What it is used for	Code
`sep / delimiter`	Separator if not comma (tab, pipe)	`sep='\t'`
`header / names`	Which row is the header / give your own names	`header=None, names=[..]`
`index_col`	Make a column the row index	`index_col='id'`
`usecols`	Read only some columns (saves memory)	`usecols=['a','b']`
`nrows / skiprows`	Read only first n rows / skip junk lines	`nrows=1000`
`na_values`	Treat extra words as missing	`na_values=['NA','-','?']`
`dtype`	Force a column's type	`dtype={'pin':str}`
`parse_dates`	Read a column as a date	`parse_dates=['date']`
`encoding`	Fix UnicodeDecodeError	`encoding='latin-1'`
`chunksize`	Read a huge file in pieces	`chunksize=100000`

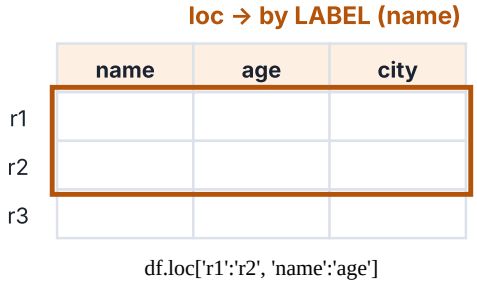
```
pd.read_excel('f.xlsx', sheet_name='Sheet1')   pd.read_json / read_sql / read_parquet
df.to_csv('out.csv', index=False)   # index=False or you get an extra column
```

2.3 Looking at Data (first 5 minutes with any dataset)

This step is called **data profiling**: how big is the data, what type is each column, how much is missing, and does anything look strange. Interviewers often ask exactly this — "what will you do first after loading the data?"

Function	What it is used for
`df.head(n) / tail(n)`	First / last n rows
`df.sample(5)`	Random rows — better than head if data is sorted
`df.shape`	(rows, columns)
`df.info()`	Column names, non-null count, dtype, memory — **most useful single command**
`df.describe()`	count, mean, std, min, 25%, 50%, 75%, max for numeric columns
`df.describe(include='object')`	Same summary for text columns (count, unique, top, freq)
`df.dtypes`	Data type of each column
`df.columns / df.index`	Column names / row labels
`df.isnull().sum()`	Missing count per column — always run this
`df.nunique()`	How many distinct values per column
`df.duplicated().sum()`	How many duplicate rows
`df.memory_usage(deep=True)`	Memory used by each column

2.4 loc vs iloc (guaranteed question)



iloc → by POSITION (0,1,2)

df.iloc[0:2, 0:2] → end NOT included

END IS INCLUDED

'r1':'r2' gives r1 AND r2

Works with a boolean mask too:

df.loc[df.age > 25, 'city'] = 'X'

Point	`loc`	`iloc`
Selects by	Label / name of row and column	Integer position (0, 1, 2...)
Slice end	**Included**	**Not included**
Boolean mask	Allowed	Not allowed (needs `.values`)
Assigning safely	`df.loc[mask,'col'] = x` — correct way	—

```
df.iloc[0]          df.iloc[0:5]        df.iloc[0:5, 1:3]      df.iloc[-1]
df.loc[2]           df.loc[2:5]         df.loc[:, 'name':'age']
df.loc[df['age'] > 25, ['name','city']]
df.at[2,'age']      df.iat[2,1]          # fastest for ONE value
```

2.5 Filtering (Boolean Masking), isin, query

How filtering actually works: `df['age'] > 25` does not give you rows — it gives a **Series of True/False**, one for every row. That is the **mask**. When you put the mask inside `df[...]`, Pandas keeps only the rows where it is True. Everything else (isin, between, str.contains) is just another way of building that same mask.

```
df[df['age'] > 25]
df[(df['age'] > 25) & (df['city'] == 'Delhi')]    # & | ~  with brackets
df[df['city'].isin(['Delhi','Pune'])]             # match a list (instead of many ORs)
df[~df['city'].isin(['Delhi'])]                   # NOT in
df[df['age'].between(20, 30)]
df[df['name'].str.contains('an', case=False, na=False)]
df[df['name'].str.startswith('A')]
df[df['city'].isnull()]
df.query('age > 25 and city == "Delhi"')          # SQL-like, easier to read
df.nlargest(3, 'salary')     df.nsmallest(3, 'salary')
```

NUMBER 1 PANDAS MISTAKE

Using `and` / `or` instead of `&` / `|`, or forgetting the brackets around each condition. `and` works on single True/False, but here every row has its own True/False, so you must use the element-wise operators.

2.6 value_counts, sort_values, drop_duplicates

`value_counts()` gives the **frequency distribution** of a category column — which value comes how many times. It is the first thing to run on any text column, because it instantly shows spelling mistakes ("Delhi" and "delhi"), rare categories, and class imbalance.

Code	What it is used for
df['city'].value_counts()	Count of each category, biggest first
.value_counts(normalize=True)	Percentage share instead of count
.value_counts(dropna=False)	Also count the missing values
.value_counts(bins=4)	Bin a numeric column into 4 ranges and count
df.sort_values('age')	Sort rows by a column (ascending by default)
sort_values(['a','b'], ascending=[True,False])	Sort by two columns in different directions
sort_values('a', na_position='first')	Where to keep the NaN rows
df.sort_index()	Sort by the index instead
df.drop_duplicates()	Remove fully duplicate rows
drop_duplicates(subset=['name'], keep='last')	Duplicate by some columns only; keep `'first'` / `'last'` / `False` (drop all)
df.duplicated()	True/False mask of duplicates — to **see** them before deleting

Real use of value_counts: checking class imbalance in a target column before training a model.

2.7 Series Operations, .str and .dt

```
s.sum() mean() median() std() min() max() count() nunique() unique()
s.idxmax()           # index label of the biggest value
s.abs() round(2) rank() cumsum() pct_change() clip(0,100)
s + 10, s * 2, s > 50          # vectorized, like NumPy
s.isnull(), s.notnull(), s.astype(int)
```

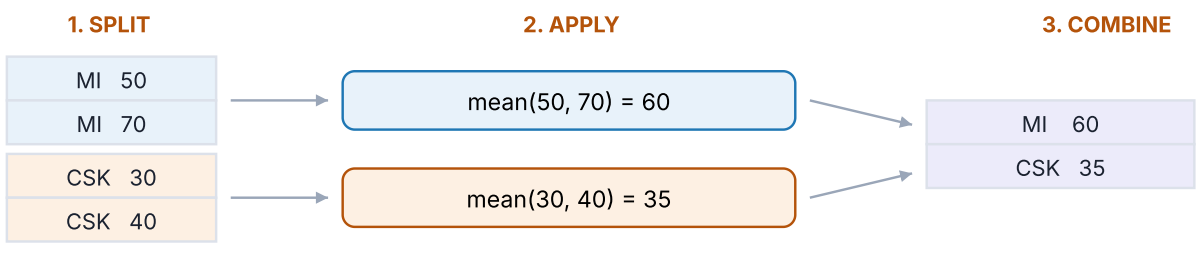
.str — text columns

.str.lower() / upper()	case
.str.strip()	remove extra spaces
.str.contains('x')	search
.str.replace('a','b')	replace text
.str.split('-')	break into parts
.str.len()	length
.str.extract(r'(\d+)')	pull out by regex

.dt — date columns

.dt.year / month / day	parts of a date
.dt.day_name()	Monday, Tuesday...
.dt.quarter	Q1..Q4
.dt.hour / minute	time parts
.dt.days (on a diff)	number of days between dates
pd.to_datetime(col)	convert text to date FIRST

2.8 groupby() (the most asked Pandas topic)



df.groupby('team')['runs'].mean() — same as SQL GROUP BY

Theory: `df.groupby('team')` alone prints only an object, not a table. That is because Pandas is **lazy** here — it has only made the groups, and nothing is calculated until you tell it *what* to calculate (`.mean()`, `.sum()`, `.agg()`).

Code	What it is used for
`df.groupby('team')['runs'].mean()`	Average per group (most common)
`df.groupby(['team','season']).sum()`	Group by two columns
`.agg(['mean','max','count'])`	Many calculations at once
`.agg(avg=('runs','mean'), total=('runs','sum'))`	Named aggregation — clean column names
`.agg({'runs':'sum','wkts':'mean'})`	Different calculation for each column
`.size()`	Rows per group (counts NaN too)
`.count()`	Non-null values per group
`.transform('mean')`	Group value put back on **every row** (same shape) — for % of group, filling NaN by group
`.filter(lambda g: len(g) > 5)`	Keep only groups that pass a condition
`.apply(func)`	Any custom function on each group (slowest)
`.first() / .last() / .nth(0)`	First / last row of each group
`groupby(..., as_index=False)`	Keep the group key as a normal column
`groupby(..., dropna=False)`	Also make a group for NaN keys

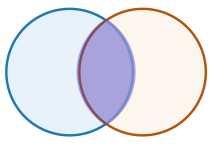
AGG VS TRANSFORM VS APPLY — KNOW THIS DIFFERENCE

agg → one row per group (shrinks the data). **transform** → same number of rows as before (broadcasts the group result back). **apply** → flexible, can return anything, but it is the slowest.

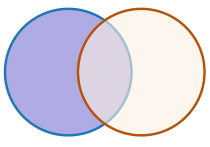
```
# classic transform use: each player's share of his team total
df['pct'] = df['runs'] / df.groupby('team')['runs'].transform('sum') * 100
```

2.9 merge, join, concat

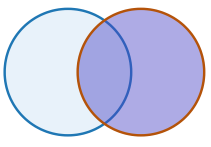
Joining means **bringing columns from a second table** next to the first one, matched on a common column (the **key**). It is exactly SQL JOIN. The only decision is: what to do with rows that have no match — and that is what `how=` answers.



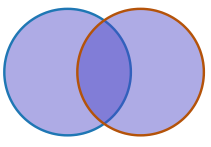
inner
only matching



left
all of left table



right
all of right table



outer
everything, NaN where missing

```
pd.merge(a, b, on='team', how='inner')          # how = inner | left | right | outer
pd.merge(a, b, left_on='id', right_on='emp_id') # different column names
pd.merge(a, b, on='team', suffixes=('_x','_y')) # same column name in both
pd.merge(a, b, on='team', indicator=True)       # shows where each row came from
a.join(b, how='left')                           # joins on the INDEX
pd.concat([a, b])                               # stack rows (axis=0)
pd.concat([a, b], axis=1)                       # stack columns side by side
pd.concat([a, b], ignore_index=True)
```

Tool	Joins on	Use for
`merge`	Columns (or index)	SQL-style joining of two tables — main one
`join`	Index by default	Quick merge when the index is the key
`concat`	Nothing — just glues	Stacking same-shaped tables (Jan + Feb data)

MERGE TRAP

If the key has duplicate values in both tables, you get **more rows than you started with** (a many-to-many join). Always check `df.shape` before and after a merge, and use `validate='one_to_one'` when you expect a clean key.

2.10 pivot_table, crosstab, melt, pivot

Reshaping means changing the **layout** of the table without changing the information. **Long** format has many rows and few columns (good for storing and for Seaborn); **wide** format spreads one column's values into many columns (good for reading and reports). `pivot` goes long → wide, `melt` goes wide → long.

```
df.pivot_table(index='team', columns='season', values='runs',
               aggfunc='mean',        # mean is the DEFAULT
               margins=True,          # adds "All" total row/column
               fill_value=0)
pd.crosstab(df['team'], df['toss_decision'])          # count table of 2 categories
pd.crosstab(df['team'], df['result'], normalize='index')   # row-wise %
df.pivot(index='date', columns='city', values='temp')      # reshape only, NO aggregation
df.melt(id_vars='month', var_name='year', value_name='sales')  # wide -> long
df.stack() / df.unstack()
```

Tool	Difference
`pivot_table`	Reshapes **and** aggregates. Handles duplicate index/column pairs.
`pivot`	Only reshapes. **Errors** if a pair repeats.
`crosstab`	Shortcut of pivot_table for counting two categories.
`melt`	Opposite of pivot: wide → long (needed for Seaborn `hue`).

Interview line: "pivot_table is the Excel pivot table of Pandas: `index` = rows, `columns` = columns, `values` = the number, `aggfunc` = how to summarise it."

2.11 corr()

Correlation answers: **when one column goes up, what does the other do?** It is a number from −1 to +1. Important theory: it only measures a **straight-line** relation, and it is **unit-free**, so you can compare salary-vs-age with height-vs-weight on the same scale.

```
df.corr(numeric_only=True)            # Pearson (linear) by default
df.corr(method='spearman')            # rank based - catches non-linear but monotonic
df['a'].corr(df['b'])
df.corr()['price'].sort_values(ascending=False)   # what relates to the target
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
```

Value	Meaning
+1	Perfect positive: one goes up, other goes up
0	No linear relation (there may still be a curved relation)
−1	Perfect negative: one goes up, other goes down

TWO LINES THAT SCORE MARKS

1. **Correlation is not causation.** 2. Pearson only measures a **straight-line** relation; a perfect U-shaped relation can give correlation 0. Also, two features with very high correlation (>0.9) mean **multicollinearity** — drop one before linear regression.

2.12 rename, set_index, reset_index

Code	What it is used for
`df.rename(columns={'old':'new'})`	Change column names (safest way)
`df.rename(index={0:'first'})`	Change row labels
`df.columns = ['a','b','c']`	Replace all names at once (must give all)
`df.rename(columns=str.lower)`	Apply a function to all names
`df.set_index('id')`	Make a column the row index — then `df.loc['x']` finds a row instantly, and merges/joins on the index become easy
`df.set_index(['a','b'])`	Two-level (MultiIndex) — for grouped data
`df.reset_index()`	Index becomes a normal column again (used after groupby)
`df.reset_index(drop=True)`	Throw the old index away — use after filtering to get 0,1,2... again

"SET_INDEX KIS KAAM AATA HAI?" — THE SHORT ANSWER

"`set_index` makes a meaningful column (like id or date) the row label. After that, lookups with `.loc` are fast and readable, joins on that key are easier, and time-series slicing like `df.loc['2021']` works. `reset_index()` undoes it — I use it after a `groupby` to turn the group keys back into normal columns."

2.13 Missing Values: dropna & fillna

Missing values are shown as `NaN` (Not a Number). Theory first: **why** is it missing? That decides what to do.

Type	Meaning	Safe to do
MCAR (completely at random)	Missing for no reason at all — a form got skipped	Dropping is safe
MAR (at random)	Missing depends on another column — e.g. young people skip "income"	Fill using that other column (groupby fill)
MNAR (not at random)	Missing depends on the hidden value itself — high earners hide income	Dangerous; dropping creates bias. Mark it as "Unknown"

Imputation = the word for filling missing values. Dropping is simple but you lose data; filling keeps the rows but can reduce the real variance.

```
df.isnull().sum()                 # count per column - ALWAYS run first
df.isnull().sum().sum()           # total missing
df['a'].isnull().mean()*100       # % missing in a column

df.dropna()                       # drop rows having ANY NaN
df.dropna(how='all')              # drop only fully empty rows
df.dropna(subset=['salary'])      # drop only if THIS column is missing
df.dropna(axis=1)                 # drop columns instead of rows
df.dropna(thresh=3)               # keep rows having at least 3 real values

df.fillna(0)
df['age'].fillna(df['age'].median())      # median is safer when outliers exist
df['city'].fillna(df['city'].mode()[0])   # mode for text columns
df.ffill()   df.bfill()                   # carry forward / backward (time series)
df['a'].interpolate()                     # estimate between two known values
df['age'] = df.groupby('city')['age'].transform(lambda s: s.fillna(s.mean()))
```

Situation	What to do
Very few missing rows	`dropna()`
Column is 60–70% empty	Drop the whole column
Numeric, no outliers	Fill with mean
Numeric, with outliers	Fill with **median**
Text / category	Fill with mode or the word "Unknown"
Time series	`ffill()` or `interpolate()`

REMEMBER

Pandas methods return a **new** object. `df.dropna()` alone changes nothing — write `df = df.dropna()`. Also, `NaN == NaN` is False, so always use `isnull()`.

2.14 Data Types & Cleaning Checklist

```
df['a'] = df['a'].astype('int32')
pd.to_numeric(df['a'], errors='coerce')     # bad text becomes NaN instead of error
pd.to_datetime(df['date'], errors='coerce', dayfirst=True)
df['city'] = df['city'].astype('category')  # big memory saving for repeated text
```

1. Check `df.info()` — are numbers stored as text? are dates stored as text?
2. `df.isnull().sum()` → decide drop or fill.
3. `df.duplicated().sum()` → `drop_duplicates()`.
4. Clean text: `.str.strip().str.lower()` (so "Delhi" and " delhi " become one category).
5. Check `value_counts()` of every category column for spelling mistakes.
6. Check outliers with `describe()` and a boxplot.
7. Rename columns to clean lowercase names.

2.15 apply, map, cut, qcut, assign, explode

These are for **feature engineering** — making a new column out of existing ones. The theory to remember: `map` and `apply` still run a Python loop inside, so they are slow; a vectorized expression or `np.where` does the same work in C and is much faster. **Binning** (cut/qcut) means turning a numeric column into categories, which helps when the exact number does not matter, only the range.

Code	What it is used for
`s.map({'M':1,'F':0})`	Replace values using a dictionary (Series only)
`s.map(lambda x: x*2)`	Run a function on every value of a Series
`df.apply(func, axis=1)`	Run a function on every **row** (axis=1) or column (axis=0)
`df.applymap(func)`	Every single cell (older name; now `df.map`)
`pd.cut(df.age, bins=[0,18,35,60], labels=['kid','young','old'])`	Make categories from ranges **you** choose
`pd.qcut(df.salary, 4, labels=['Q1','Q2','Q3','Q4'])`	Make **equal-sized** groups (quartiles)
`df.assign(bonus=df.salary*0.1)`	Add a column and keep chaining
`df.explode('tags')`	A list inside a cell becomes one row per item
`np.where(df.age>30,'old','young')`	Fastest if-else for a new column
`df.replace({'M':'Male'})`	Replace values anywhere

CUT VS QCUT

`cut` = equal **ranges** (bins you decide), group sizes can be very unequal. `qcut` = equal **counts** in each group (quantiles), ranges will be unequal.

Speed order: vectorized / `np.where` > `map` > `apply` > `iterrows`. Never use `iterrows` if a vectorized way exists — this is a favourite performance question.

2.16 Rolling, Time Series, Copy Rules

Daily data jumps up and down a lot (noise). A **rolling / moving average** takes the average of the last N values and slides forward, which smooths that noise and shows the real trend — this is why every stock chart has a 7-day or 50-day moving average. **Resampling** means changing the time level, for example daily data to monthly totals.

```
df['ma7'] = df['sales'].rolling(window=7).mean()   # 7-day moving average
df['sales'].rolling(3).sum()    df['sales'].expanding().mean()
df['growth'] = df['sales'].pct_change()*100
df['prev'] = df['sales'].shift(1)                  # previous row's value

df = df.set_index('date')
df.loc['2021']            df.loc['2021-01':'2021-06']
df.resample('M').sum()    # monthly total ('D' day, 'W' week, 'Q' quarter, 'Y' year)
```

SETTINGWITHCOPYWARNING — WHAT AND WHY

It appears with **chained assignment** like `df[df.a > 1]['b'] = 0`. Pandas does not know whether the first part gave a view or a copy, so the change may be lost. Correct way: **`df.loc[df.a > 1, 'b'] = 0`**. If you want a separate table, write `new = df[mask].copy()`.

2.17 Pandas vs SQL + Performance

SQL	Pandas
SELECT col1, col2 FROM t	`df[['col1','col2']]`
WHERE age > 25	`df[df.age > 25]`
ORDER BY age DESC	`df.sort_values('age', ascending=False)`
GROUP BY city	`df.groupby('city')`
HAVING count > 5	`.filter(lambda g: len(g) > 5)`
JOIN ON id	`pd.merge(a, b, on='id')`
UNION ALL	`pd.concat([a, b])`
DISTINCT	`df.drop_duplicates()` / `.unique()`
LIMIT 10	`df.head(10)`
COUNT(*) GROUP BY	`df['c'].value_counts()`

Performance tips: use `usecols` and `dtype` in read_csv; convert repeated text columns to `category`; downcast `int64` → `int32`; avoid `apply` / `iterrows`; use `chunksize` for huge files; save as Parquet instead of CSV.

2.18 df.plot() — quickest chart

```
df['sales'].plot(kind='line')          # line | bar | barh | hist | box | kde | area | pie | scatter
df.plot(kind='scatter', x='age', y='salary')
df['city'].value_counts().plot(kind='bar')
df.groupby('team')['runs'].mean().plot(kind='barh', figsize=(8,4), title='Avg runs')
df.plot(subplots=True, layout=(2,2))
```

It is a Matplotlib wrapper, so `plt.title()`, `plt.savefig()` and `ax=` all work with it. Use it for a fast look; use Seaborn when you need statistics or `hue`.

PART 3

Matplotlib

The drawing engine. Full control over every line, tick and colour.

3.1 Architecture + Figure / Axes / Axis

THEORY: WHY DO WE NEED CHARTS AT ALL?

Numbers hide patterns. The famous proof is **Anscombe's Quartet** — four datasets with the **same mean, same variance and same correlation**, but when you plot them, one is a straight line, one is a curve, one has an outlier and one is a vertical cluster. Summary statistics alone can lie; a chart shows the truth. That is the standard answer to "why visualization?"

3 layers:

1. **Backend** — actually renders (Agg for PNG, PDF, TkAgg for windows).
2. **Artist** — every visible object: Figure, Axes, Line2D, Text, Tick.
3. **Scripting** — `pyplot`, the simple interface we write.

Term	Simple meaning
Figure	The whole page / canvas
Axes	One plot on that page (= one subplot)
Axis	The x or y number line of that plot
Artist	Anything you can see
Spines	The 4 border lines of the box

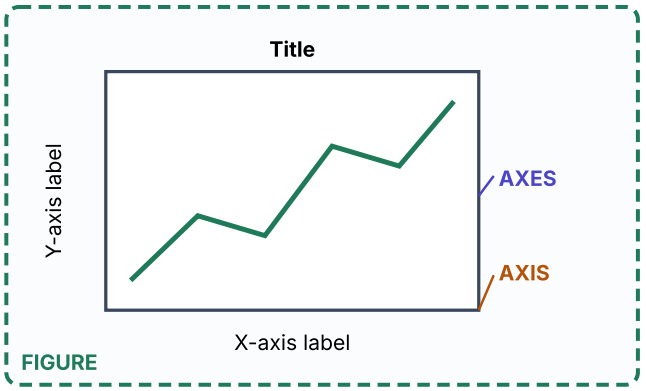


Figure > Axes > Axis

TRAP

"Axes" is not the plural of "Axis." Axes = one full plot area. Axis = one number line. One-liner: "Figure is the paper, Axes is the graph on it, Axis is the ruler on the side."

3.2 pyplot vs Object-Oriented

pyplot (state machine)

```
plt.figure(figsize=(8,4))
plt.plot(x, y)
plt.title('Sales')
plt.xlabel('Month')
plt.show()
```

Object-Oriented (recommended)

```
fig, ax = plt.subplots(figsize=(8,4))
ax.plot(x, y)
ax.set_title('Sales')
ax.set_xlabel('Month')
plt.show()
```

Point	pyplot	Object-Oriented
State	Hidden "current axes"	You hold `ax` yourself
Many subplots	Confusing	Clean: `ax[0]`, `ax[1]`
Inside functions	Risky	Safe — pass `ax` as an argument
Naming	`plt.title()`	`ax.set_title()`
Use for	Quick EDA	Dashboards, reports, production

Rule: almost every `plt.xyz()` becomes `ax.set_xyz()`; plotting functions (`plot`, `scatter`, `legend`) keep the same name.

3.3 All Chart Types + When to Use

Theory behind chart choice: a chart turns a number into something the eye can see — a position, a length, an angle, an area, a colour or a size. These are called **encoding channels**. The eye reads **position and length very accurately**, but angle, area and colour only roughly. That single fact explains why bar and scatter charts are trusted and why pie charts and 3-D charts are discouraged.

Function	Use when	Key parameters
`plt.plot(x,y)`	Trend over a continuous/time axis	`color, ls, lw, marker, label`
`plt.scatter(x,y)`	Relation between 2 numeric columns; outliers	`c, s, cmap, alpha`
`plt.bar / barh`	Compare a value across categories	`width, bottom, color`
`plt.hist(v)`	Distribution of ONE numeric column	`bins, density, cumulative`
`plt.pie(v)`	Share of a whole (only 2–5 slices)	`autopct, explode, startangle`
`plt.boxplot(d)`	5-number summary + outliers, compare groups	`vert, showmeans, notch`
`plt.violinplot(d)`	Distribution shape + summary	`showmedians`
`plt.stackplot(x,y1,y2)`	Part-to-whole over time (area chart)	`labels`
`plt.fill_between(x,lo,hi)`	Shade a confidence / min-max band	`alpha`
`plt.errorbar(x,y,yerr=e)`	Point with error bars	`capsize`
`plt.hexbin(x,y)`	Huge scatter data (density)	`gridsize`
`plt.stem / step`	Discrete values / sudden changes	–
`plt.imshow(mat)`	Image or matrix as a heatmap	`cmap, vmin, vmax, interpolation`

Format string shortcut & main style keywords

```
plt.plot(x, y, 'ro--')   # r=red, o=circle marker, --=dashed
# colors: b g r c m y k w  | markers: . o s ^ * + x D  | lines: - -- -. :
plt.plot(x, y, color='#1f77b4', linestyle='--', linewidth=2, marker='o',
         markersize=7, alpha=.8, label='2024')
plt.title('t'); plt.xlabel('x'); plt.ylabel('y'); plt.legend(loc='best')
plt.xlim(0,12); plt.xticks(rotation=45); plt.grid(alpha=.4); plt.show()
```

Chart selection — the answer they want

You want to show	Chart
Trend over time	Line, area
Compare categories	Bar / horizontal bar
Distribution (1 numeric)	Histogram, KDE, box
Relation (2 numeric)	Scatter
Part of a whole	Stacked bar, pie/donut
Correlation of many columns	Heatmap of `df.corr()`
Spread across groups	Box / violin

HISTOGRAM VS BAR CHART (ASKED EVERY TIME)

Histogram → x-axis is **numeric ranges (bins)**, bars touch, y is the count, order cannot change. Bar chart → x-axis is **categories**, bars have gaps, order can change. Histogram answers "how is this column distributed", bar answers "which category is bigger".

THREE MORE QUICK ONES

• Bar chart y-axis **must start at 0**, else it misleads. • Pie charts are discouraged — people judge length better than angle; use a horizontal bar. • `plt.plot(x,y,'o')` is faster than `scatter` but all dots must look the same; `scatter` allows per-point colour and size.

3.4 Subplots & Layout

Putting many small charts side by side is called **small multiples**. It works because the reader learns the chart once and then only compares the shapes. Important rule: keep the same scale in all panels (`sharey=True`), otherwise the comparison is not fair.

```
fig, ax = plt.subplots(2, 2, figsize=(12,8), sharex=True, sharey=True)
ax[0,0].plot(x,y);  ax[0,1].scatter(x,y);  ax[1,0].hist(d);  ax[1,1].bar(c,v)
fig.suptitle('Dashboard')        # title of the WHOLE figure
plt.tight_layout()               # fix overlapping labels

plt.subplot(2,2,1)               # old style, index starts at 1
fig.add_subplot(2,2,1)
fig.add_axes([0.1,0.1,0.8,0.8])  # inset plot (left,bottom,width,height)

import matplotlib.gridspec as gridspec          # unequal sizes
gs = gridspec.GridSpec(2, 2, height_ratios=[2,1])
ax1 = fig.add_subplot(gs[0, :])  # spans the full top row
plt.subplot2grid((3,3),(1,0), rowspan=2, colspan=2)
```

`figsize=(w,h)`	`size in inches — pixels = figsize × dpi`
`sharex / sharey`	`same scale for all subplots (fair comparison)`
`tight_layout()`	`auto spacing; newer: constrained_layout=True`
`hspace / wspace`	`gap between subplots`
`fig.suptitle vs ax.set_title`	`whole figure vs one subplot`

With `nrows=1` or `ncols=1`, `ax` is a **1-D** array (`ax[0]`); with 1×1 it is a single object, not an array — that is the `'AxesSubplot' object is not subscriptable` error.

3.5 Colours, Styles, Annotation, Axis Control

Colour & colormap

```
color='red' / 'r' / '#1f77b4' / (.1,.4,.8) /
'C0' / 'tab:blue'
cmap: sequential  viridis, Blues, plasma
      diverging   coolwarm, RdBu  (centre = 0)
      qualitative tab10, Set1     (categories)
```

Global style

```
plt.style.use('ggplot')     # fivethirtyeight,
dark_background
plt.rcParams['figure.figsize'] = (10,5)
plt.rcParams['font.size'] = 12
plt.rcdefaults()
```

Text & annotation

```
plt.text(5, 20, 'Peak', fontsize=12)
plt.annotate('High', xy=(5,25), xytext=(7,30),
   arrowprops=dict(arrowstyle='->'))
plt.axhline(50, color='r', ls='--')
plt.axvline(10);  plt.axhspan(40,60, alpha=.1)
plt.legend(loc='best', ncol=2, frameon=False,
           bbox_to_anchor=(1.02,1))  # outside
```

Axis control

```
ax.set_xlim(0,100);  ax.set_yscale('log')
ax.set_xticks(t); ax.set_xticklabels(l)
ax.tick_params(axis='x', labelsize=9)
ax.spines['top'].set_visible(False)
ax2 = ax.twinx()   # 2nd y-axis, shared x
```

text vs annotate: `text` only writes; `annotate` writes **and draws an arrow** to a data point. **Legend empty?** you forgot `label=`. **Why viridis not jet?** viridis is perceptually uniform and colour-blind friendly.

3.6 3D, Saving, Backends, Animation

```
# 3D
ax = fig.add_subplot(projection='3d')
ax.scatter3D(x,y,z, c=z, cmap='viridis');  ax.plot_surface(X, Y, Z)
X, Y = np.meshgrid(x, y)        # 1-D -> 2-D grids, then Z = f(X,Y)
ax.view_init(elev=30, azim=45)

# Saving (ALWAYS before plt.show())
plt.savefig('chart.png', dpi=300, bbox_inches='tight', transparent=True)
# PNG/JPG = raster (blurs on zoom) | SVG/PDF = vector (sharp) for reports

# Backends / Jupyter
%matplotlib inline      # static image in notebook
%matplotlib widget      # interactive
matplotlib.use('Agg')   # server with no display; fixes "no display name"
plt.close('all')        # free memory in loops

# Animation
from matplotlib.animation import FuncAnimation
line, = ax.plot([], [])
def update(frame):
    line.set_data(x, np.sin(x + frame/10));  return line,
ani = FuncAnimation(fig, update, frames=100, interval=50, blit=True)
ani.save('wave.gif', writer='pillow')
```

CLASSIC BUG

Saved image is blank → you called `savefig()` **after** `show()`. `show()` clears the figure. Save first, show after.

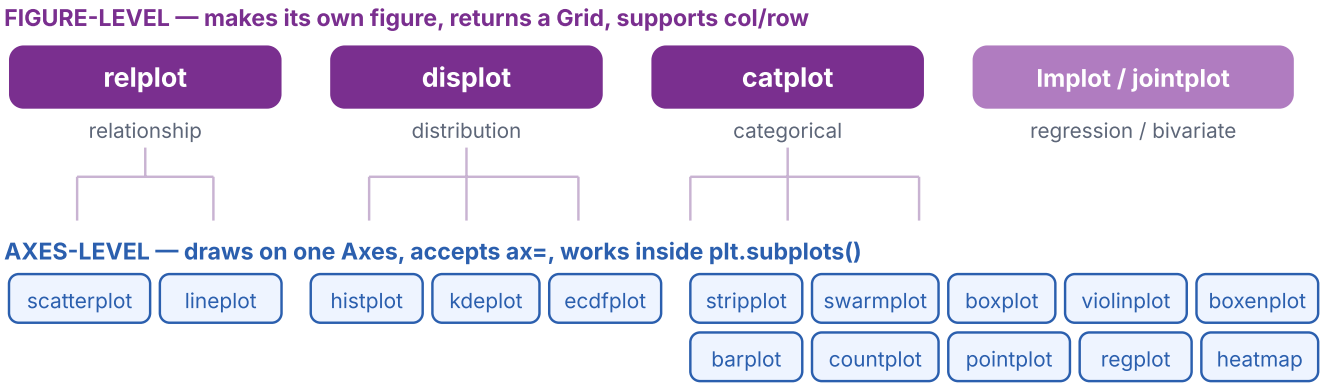
Why meshgrid? Z must be computed at every (x,y) pair. meshgrid turns two 1-D arrays into two 2-D grids so `Z = f(X,Y)` is one vectorized step. Same trick draws ML decision boundaries.

PART 4

Seaborn

Statistical charts in one line, straight from a DataFrame. Built on Matplotlib, not a replacement for it.

4.1 Figure-level vs Axes-level (the #1 Seaborn question)



Each figure-level function is a wrapper over its axes-level family — you choose with kind=

Point	Figure-level	Axes-level
Returns	`FacetGrid` / Grid object	Matplotlib `Axes`
`ax=`	**Not allowed** (TypeError)	Allowed
Size	`height` + `aspect`	`figsize` in `plt.subplots()`
`col` / `row`	Yes — its main benefit	No
Legend	Outside the plot	Inside
Customize	`g.set_axis_labels()`, `g.fig`	`ax.set_*()`

IN INTERVIEW, SAY THIS

"Axes-level functions draw on one Matplotlib Axes and accept `ax=`, so they fit inside my own subplot grid. Figure-level functions — relplot, displot, catplot — create and own the whole figure, return a FacetGrid, and support `col` / `row` faceting. One panel inside my layout → axes-level. Many panels split by a category → figure-level."

This single idea explains two common errors: `figsize` having no effect, and `ax=` throwing a TypeError.

4.2 Long vs Wide Data

Seaborn wants **long / tidy** data: **one row = one observation**, one column = one variable. Only then can `hue`, `col`, `row` take a column name.

```
# WIDE:  month | 2023 | 2024      (year is hidden inside column names)
# LONG:  month | year | sales     (now hue='year' works)
long_df = wide_df.melt(id_vars='month', var_name='year', value_name='sales')
sns.lineplot(data=long_df, x='month', y='sales', hue='year')
```

Datasets to practise on: `sns.load_dataset('tips' / 'titanic' / 'iris' / 'flights')`.

4.3 All Plots, Family by Family

Seaborn groups its plots into three families, and the family depends only on the **data type**: two numeric columns → **relational**; a category with a number → **categorical**; one numeric column alone → **distribution**. If you remember this, you can name the right function without memorising the list.

Relational (2 numeric columns)

Function	What it is used for	Key parameters
`sns.scatterplot`	Relation between 2 numeric columns	`hue, style, size, sizes`
`sns.lineplot`	Trend; if x repeats it plots the **mean + 95% CI band**	`estimator, errorbar=None`
`sns.relplot`	Figure-level version of both	`kind, col, row, height, aspect`

Categorical (category vs number)

`sns.stripplot`	Every point, with random **jitter** — points can still overlap, fast	`jitter, dodge, alpha`
`sns.swarmplot`	Every point, **no overlap** (calculated) — shows density, slow on big data	`dodge, size`
`sns.boxplot`	5-number summary + outliers; compare groups	`hue, showfliers, orient`
`sns.violinplot`	Full shape (KDE) + box inside; shows 2 peaks which box cannot	`split=True, inner, cut`
`sns.boxenplot`	Boxplot for **large** data — shows tails better	–
`sns.barplot`	Mean (or other estimator) per category + CI error bar	`estimator, errorbar`
`sns.countplot`	Count of rows per category (= value_counts)	`hue, order`
`sns.pointplot`	Mean as points joined by a line — shows **interaction** between 2 categories	`dodge, markers`
`sns.catplot`	Figure-level wrapper for all of the above	`kind, col, row`

Distribution (1 numeric column)

`sns.histplot`	Histogram; x and y together gives a 2-D histogram	`bins, kde, multiple, stat`
`sns.kdeplot`	Smooth density curve (area = 1); 2 columns gives contours	`fill, bw_adjust, cut`
`sns.ecdfplot`	Cumulative — no bins, no smoothing, nothing hidden	`hue`
`sns.rugplot`	Small ticks for each observation, added under a KDE	`height`
`sns.displot`	Figure-level: kind = hist / kde / ecdf	`kind, col`

THEORY: KDE AND THE CONFIDENCE INTERVAL BAND

KDE (Kernel Density Estimate) is a smooth histogram. Instead of putting values into bins, it places a small bell curve (the kernel) on every data point and adds them all up. The total area is 1, so the y-axis is **density**, not count. `bw` (bandwidth) controls smoothness — small bw gives a spiky curve, large bw hides the real shape.

The shaded band in `lineplot` and the black line on `barplot` are both a **95% confidence interval**: Seaborn resamples the data many times (bootstrapping) and shows the range where the true mean most likely lies. A wide band means few samples or high variation, so do not trust that mean too much.

DISTPLOT IS REMOVED

Old `sns.distplot()` is deprecated and removed. Use `sns.histplot(x, kde=True)` (axes-level) or `sns.displot(x, kde=True)` (figure-level). Saying this scores points.

THREE COMPARISONS THEY ALWAYS ASK

strip vs swarm: random jitter (can overlap, fast) vs computed positions (no overlap, slow).
box vs violin: summary + outliers vs full KDE shape; **box cannot show a bimodal distribution**.
bar vs count: barplot needs x **and** y and shows the mean; countplot needs only x and shows the row count.
The black line on a barplot is the **95% confidence interval of the mean**.

4.4 heatmap, clustermap, joint, pair, FacetGrid

```
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt='.2f',
            cmap='coolwarm', vmin=-1, vmax=1, linewidths=.5,
            mask=np.triu(np.ones_like(df.corr(), dtype=bool)))   # hide duplicate half
sns.clustermap(df, standard_scale=1, cmap='viridis')  # reorders rows/cols + dendrogram

sns.jointplot(data=tips, x='total_bill', y='tip', kind='hex')   # scatter|kde|hex|reg
sns.pairplot(iris, hue='species', diag_kind='kde', corner=True, vars=['a','b'])

g = sns.FacetGrid(tips, col='time', row='smoker', hue='sex', height=3.5)
g.map(sns.scatterplot, 'total_bill', 'tip');  g.add_legend()
```

Tool	What it shows	Level
`heatmap`	Matrix as colours, in the given order	Axes
`clustermap`	Same + hierarchical clustering reorders rows/columns, adds dendrograms	Figure
`jointplot`	Bivariate centre + each variable's distribution on the margins	Figure (JointGrid)
`pairplot`	Every numeric column vs every other; diagonal = its own distribution	Figure (PairGrid)
`FacetGrid`	Same plot repeated per category (small multiples)	Figure
`regplot` / `lmplot`	Scatter + regression line + CI band (lmplot adds hue/col)	Axes / Figure
`residplot`	Residuals — a pattern means linear model is not suitable	Axes

pairplot limit: with 20 columns it draws 400 tiny plots. Use a correlation heatmap first, then pairplot only the selected columns with `vars=`. **Why scale in clustermap?** clustering uses distance, so a column in thousands would dominate one in decimals.

4.5 Styling & Palettes

```
sns.set_theme(style='whitegrid', context='talk', palette='Set2', font_scale=1.1)
sns.despine()          # remove top & right spines
sns.reset_defaults()
```

Setting	Options
style (5)	`darkgrid` (default), `whitegrid`, `dark`, `white`, `ticks`
context (4)	`paper` < `notebook` (default) < `talk` < `poster` — scales fonts/lines
palette	qualitative: `deep, muted, pastel, colorblind, Set2` · sequential: `Blues, rocket, viridis` · diverging: `coolwarm, vlag`

```
sns.color_palette('husl', 8)   sns.set_palette('pastel')   sns.palplot(sns.color_palette())
```

SEABORN GOTCHAS

• `figsize` ignored → figure-level function; use `height` / `aspect`. • `ax=` TypeError → use the axes-level version. • swarmplot "points cannot be placed" → too many points; use stripplot. • `ci=` is now `errorbar=`. • heatmap on text columns → add `numeric_only=True`. • `hue` not working → data is wide; `melt()` it first.

PART 5

Rapid-Fire Interview Q&A

88 real questions in two rounds. Short answers, the way you can actually speak them.

▸ NumPy

1. What is NumPy and why is it faster than a Python list?

NumPy gives the ndarray — a fixed-type, n-dimensional array. It is faster because the loops run in compiled C instead of Python, and because the numbers of one type sit together in memory. This is called vectorization.

2. List vs array — what happens with `a + b`?

For lists it joins them; for arrays it adds item by item. Lists allow mixed types and can grow; arrays have one type and a fixed size but have all the maths tools.

3. Difference between arange and linspace?

arange takes a **step** and excludes the end; linspace takes **how many numbers** you want and includes the end. For plotting I use linspace because the count is guaranteed.

4. What does axis=0 and axis=1 mean?

axis is the direction that disappears. axis=0 goes down the rows and gives one value per column; axis=1 goes across the columns and gives one value per row.

5. Does slicing a NumPy array give a copy?

No, it gives a **view** — same memory. Changing the slice changes the original. Python lists copy on slicing, so this surprises people. Use `.copy()` for an independent array; fancy and boolean indexing always copy.

6. ravel vs flatten?

Both make the array 1-D. `ravel()` returns a view (faster, shares memory), `flatten()` returns a copy (safe).

7. What is broadcasting? What are its rules?

It lets arrays of different shapes work together by stretching the smaller one — without really copying memory. Rule: compare shapes from the right; each pair must be equal or one of them must be 1, else ValueError.

8. Fancy indexing vs boolean indexing?

Fancy indexing picks by a list of positions in any order; boolean indexing picks by a condition mask. Both return copies.

9. What does np.where do?

It is if-else on the whole array: `np.where(cond, value_if_true, value_if_false)`. With only the condition it returns the positions where it is True.

10. `a * b` vs `a @ b`?

`*` is element-wise multiplication; `@` (or np.dot) is matrix multiplication, where inner dimensions must match.

11. How do you handle NaN in NumPy?

`np.isnan()` to find them (`== np.nan` never works because NaN is not equal to itself), and `np.nanmean` / `np.nansum` to calculate while ignoring them.

12. How do you reduce memory of an array?

Use a smaller dtype, for example `astype('float32')` instead of float64 — that halves the memory. `a.nbytes` shows the total.

▸ Pandas

13. Series vs DataFrame?

Series is one labelled column (1-D). DataFrame is a full table (2-D) made of many Series sharing the same index.

14. Why Pandas when NumPy exists?

NumPy arrays hold one type and have no labels. A DataFrame holds different types per column, has column names and row labels, and has ready tools for missing data, grouping, joining and reading files.

15. loc vs iloc?

loc selects by label and the slice end is **included**; iloc selects by integer position and the end is **excluded**. loc also accepts a boolean mask, which is why `df.loc[mask, 'col'] = x` is the safe way to assign.

16. What are the first 5 commands you run on a new dataset?

`df.shape`, `df.head()`, `df.info()`, `df.describe()` and `df.isnull().sum()`. Then `value_counts()` on the category columns.

17. How do you filter rows on two conditions?

`df[(df.age > 25) & (df.city == 'Delhi')]` — element-wise `&` and `|` with brackets around each condition. Plain `and` / `or` fails because each row has its own True/False.

18. What is the use of isin()?

It checks membership in a list, so one line replaces many OR conditions: `df[df.city.isin(['Delhi','Pune'])]`. Put `~` in front for "not in".

19. Explain groupby.

Split-apply-combine: split the rows into groups by a key, apply a calculation to each group, and combine the results into one table. It is the same as SQL GROUP BY.

20. agg vs transform vs apply?

agg gives one row per group (data shrinks). transform gives back the same number of rows, so the group result sits on every row — good for percentage of group or filling NaN by group. apply is flexible but slowest.

21. count() vs size() in groupby?

count() counts non-null values per column; size() counts all rows in the group including NaN.

22. merge vs join vs concat?

merge joins on columns like SQL; join works on the index; concat just stacks tables, rows or columns, without matching keys.

23. Types of joins?

inner keeps only matching keys, left keeps all rows of the left table, right keeps all of the right, outer keeps everything and fills NaN where there is no match.

24. After merging I got more rows than before. Why?

Because the key is duplicated in both tables, so it became a many-to-many join. I check shape before and after, and use `validate='one_to_one'` when the key should be unique.

25. pivot_table vs pivot vs crosstab?

pivot only reshapes and errors on duplicate index/column pairs. pivot_table reshapes **and** aggregates (default mean). crosstab is a shortcut of pivot_table for counting two categories.

26. What does set_index do and when do you use it?

It makes a column the row label. After that `.loc` lookups are fast and readable, joins on that key are easier, and date slicing like `df.loc['2021']` works. `reset_index()` turns it back into a normal column — I use it after groupby.

27. How do you handle missing values?

First measure with `isnull().sum()`. If very few rows, drop them. If a column is mostly empty, drop the column. Else fill: mean for clean numeric data, **median** when outliers exist, mode or "Unknown" for text, and ffill or interpolate for time series.

28. Why mean is risky for filling?

Because outliers pull the mean. With a few very large salaries, the mean is not a typical value, so the median is safer.

29. What is SettingWithCopyWarning?

It comes from chained assignment like `df[df.a>1]['b']=0`, where Pandas cannot tell if the first step gave a view or a copy, so the change may be lost. The correct way is `df.loc[df.a>1,'b']=0`.

30. apply vs map vs vectorized operations?

map works on a Series with a dict or a function; apply works on rows or columns; both run a Python loop internally. A vectorized operation or `np.where` is much faster, so I use apply only when there is no vectorized way.

31. cut vs qcut?

cut makes bins of equal **ranges** that you define; qcut makes bins with an equal **number of rows** (quantiles).

32. How do you handle a CSV file too big for memory?

Read only needed columns with `usecols`, set smaller dtypes, use `chunksize` to process it in pieces, convert repeated text to `category`, or switch to Parquet.

33. value_counts vs groupby.count?

value_counts works on one Series and sorts by frequency; groupby is more general and can aggregate many columns at once. For class imbalance I use `value_counts(normalize=True)`.

34. What does corr() tell you and what is its limit?

It measures the linear relation between numeric columns, from −1 to +1. Limits: it only sees straight-line relations, and correlation is not causation. Very high correlation between two features means multicollinearity.

▸ Matplotlib

35. Figure vs Axes vs Axis?

Figure is the whole canvas, Axes is one plot drawn on it, and Axis is the x or y number line of that plot. One figure can hold many Axes; each Axes has two Axis objects.

36. pyplot vs object-oriented interface?

pyplot is a state machine that remembers the current figure, so commands apply implicitly. The OO interface gives me Figure and Axes objects, which is safer for multiple subplots and reusable functions.

37. Histogram vs bar chart?

Histogram is for one numeric column with ranges on the x-axis and bars touching; bar chart is for categories with gaps and a reorderable order. Histogram shows distribution, bar shows comparison.

38. How do you plot 4 charts in one figure?

`fig, ax = plt.subplots(2,2, figsize=(12,8))`, draw on `ax[0,0]` to `ax[1,1]`, then `plt.tight_layout()`. Add `sharey=True` if they must be compared on the same scale, and GridSpec if the panels need different sizes.

39. My saved image is blank. Why?

Because `savefig()` was called after `plt.show()`, and show clears the figure. Save first, then show.

40. Which colormap do you choose and why?

Sequential like viridis for low-to-high values, diverging like coolwarm when zero is a meaningful centre such as correlation, and qualitative like tab10 for categories. I avoid jet because it is not perceptually uniform and is bad for colour-blind users.

▸ Seaborn & general

41. Seaborn vs Matplotlib?
Seaborn is built on Matplotlib. It works directly with DataFrames, does the statistics itself and gives complex charts in one line; Matplotlib gives low-level control. I use both — Seaborn for the chart, Matplotlib to fine-tune it.

42. Figure-level vs axes-level in Seaborn?
Axes-level functions draw on one Axes and accept `ax=`. Figure-level functions (relplot, displot, catplot) create their own figure, return a FacetGrid and support col/row faceting, but do not accept `ax=` and are sized with height and aspect.

43. boxplot vs violinplot?
Boxplot shows the five-number summary and outliers. Violin shows the full distribution shape with a KDE plus a small box inside. The key point: a boxplot cannot reveal a bimodal distribution, the violin can.

44. How do you check correlation of all features and what do you do with it?
`sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', vmin=-1, vmax=1)`. I look at which features relate to the target, and drop one of any pair with correlation above about 0.9 to avoid multicollinearity.

45. Walk me through your EDA process.
Load with read_csv, then shape, info, describe and isnull().sum(). Clean: fix dtypes, missing values, duplicates, messy text. Then univariate charts (histplot, countplot, boxplot for outliers), then bivariate (scatterplot, boxplot by category, correlation heatmap), then multivariate with hue or faceting and pairplot on the important columns. Finally I write down the findings that matter for the business question.

5B More Rapid-Fire Questions (46–88)

Short questions, short answers. Read these out loud once — that is enough.

▸ NumPy — extra

46. What is an ndarray?
A grid of values of the **same type**, with a fixed shape, stored together in memory. "nd" means n-dimensional.

47. Why must all items have the same type?
Because then every item takes the same number of bytes, so NumPy can jump straight to any item and run the loop in C without checking types.

48. What does reshape(-1) mean?
−1 means "you figure out this number". `a.reshape(2,-1)` keeps 2 rows and NumPy calculates the columns.

49. Difference between np.zeros and np.empty?
zeros fills with 0; empty does not fill anything, so it holds garbage values but is faster. Use empty only when you will overwrite everything.

50. How do you find the position of the maximum value?
`a.argmax()`. `a.max()` gives the value, `argmax` gives the index.

51. How do you get unique values with their counts?
`np.unique(a, return_counts=True)` — it returns the unique values and how many times each one appears.

52. What is np.linspace used for in real work?
Making the x-values for a plot, because you get exactly the number of points you asked for, including the end point.

53. Why do we set a random seed?
So the same random numbers come every time. Without it, results change on every run and nobody can reproduce your work.

54. What does np.clip do?
It forces values into a range — anything below the minimum becomes the minimum, anything above the maximum becomes the maximum. Used to cap outliers.

55. How do you normalize every column of a 2-D array in one line?
`(a - a.mean(axis=0)) / a.std(axis=0)` — broadcasting applies each column's mean and std down that column.

▸ Pandas — extra

56. Difference between `df['a']` and `df[['a']]`?
Single brackets give a **Series** (one column), double brackets give a **DataFrame** (a table with one column).

57. What does inplace=True do, and do you use it?
It changes the DataFrame directly instead of returning a new one. I avoid it — it makes chaining impossible, it is being deprecated, and `df = df.something()` is clearer.

58. How do you add a new column?
`df['new'] = df['a'] * 2`, or `df.assign(new=...)` if I want to keep chaining, or `np.where(cond, x, y)` for an if-else column.

59. How do you delete a column or a row?
`df.drop(columns=['a'])` for a column and `df.drop(index=[0,1])` for rows. Remember to assign the result back.

60. How do you rename columns?
`df.rename(columns={'old':'new'})`. Assigning `df.columns = [...]` also works but you must give every name.

61. How do you find and remove duplicate rows?
`df.duplicated().sum()` to count them, then `df.drop_duplicates(subset=['id'], keep='first')`. I always look at them before deleting.

62. How do you get the top 3 salaries?
`df.nlargest(3, 'salary')`, or `df.sort_values('salary', ascending=False).head(3)`.

63. How do you find the top earner of each department?
`df.sort_values('salary', ascending=False).groupby('dept').head(1)`, or use `idxmax` with `.loc`.

64. How do you count missing values per column?
`df.isnull().sum()`. For percentage: `df.isnull().mean()*100`.

65. What is the difference between `df.size` and `df.shape`?
`shape` gives (rows, columns); `size` gives the total number of cells, that is rows × columns.

66. What does `axis=1` mean in `df.drop(..., axis=1)`?
It means work along the columns, so it drops a column. Same axis rule as NumPy: axis=0 is rows, axis=1 is columns.

67. How do you convert a column to date and then get the month?
`df['d'] = pd.to_datetime(df['d'])` and then `df['d'].dt.month`. Without converting first, `.dt` will not work.

68. How do you combine first_name and last_name?
`df['full'] = df['first'] + ' ' + df['last']` — string columns add up directly.

69. What is a MultiIndex?

An index with more than one level, which you get after grouping by two columns. `reset_index()` turns those levels back into normal columns.

70. How do you apply a function to every row?

`df.apply(func, axis=1)`. But I first check whether a vectorized version exists, because apply is slow.

71. How do you read only 1000 rows of a huge file to test your code?

`pd.read_csv('f.csv', nrows=1000)`.

72. Your merge produced NaN values. What happened?

Some keys had no match on the other side and I used a left or outer join. I check the key spelling and spaces, and use `indicator=True` to see which rows matched.

▸ Statistics & EDA

73. Mean, median or mode — which one and when?

Mean for clean numeric data, median when outliers or skew exist, mode for categories or the most common value.

74. What is skewness and how do you see it in a chart?

It means the distribution is tilted to one side. In a histogram, a long tail on the right is right/positive skew (like income) and the mean sits to the right of the median.

75. How do you detect outliers?

The IQR rule — anything below Q1 − 1.5×IQR or above Q3 + 1.5×IQR — shown directly by a box plot. The z-score method (beyond about 3 standard deviations) is the other way.

76. Should you always remove outliers?

No. First check if it is a data entry error or a genuine rare case. A real outlier like one huge order is valuable information, so I may cap it instead of deleting it.

77. Variance vs standard deviation?

Both measure spread. Variance is the average squared distance from the mean, so its unit is squared; standard deviation is its square root, so it is in the same unit as the data and easier to explain.

78. What does a bimodal histogram tell you?

That two different groups are mixed inside one column — for example male and female heights together. I would split the data by that category.

79. What is normalization vs standardization?

Normalization squeezes values into 0 to 1 using min-max. Standardization makes the mean 0 and the standard deviation 1 using the z-score. Both are done so that one large-valued column does not dominate the model.

80. What is class imbalance and how do you spot it?

One category has far more rows than the other, like 95% "no fraud". I spot it with `value_counts(normalize=True)` or a countplot. If it is ignored, the model looks accurate but learns nothing.

▸ Charts — extra

81. Which chart would you use to compare sales of 5 cities?

A bar chart, horizontal if the city names are long. One categorical column with one numeric value — that is a bar chart.

82. Which chart shows sales trend over 24 months?

A line plot, because the x-axis is time and ordered. I would add a rolling average line if the data is noisy.

83. How would you compare salary distribution across 4 departments?

A boxplot with department on x and salary on y, or a violin plot if I also want the shape. I may overlay a stripplot to show how many employees each group has.

84. What is a bubble chart?

A scatter plot where the dot size shows a third numeric variable, done with the `s` parameter in `plt.scatter` or `size=` in Seaborn.

85. How do you make a chart readable for a non-technical audience?

A title that states the finding, labelled axes with units, few colours, no 3-D effects, bars starting at 0, and the key number written directly on the chart.

▸ Scenario questions

86. You are given a messy sales CSV. What are your first 10 minutes?

Load it, check shape, head, info and describe. Count missing values and duplicates. Fix the data types, especially dates and numbers stored as text. Clean the text columns with strip and lower, then check value_counts for spelling mistakes. Only after that I start analysing.

87. Your groupby result has fewer rows than expected. Why?

Most likely the grouping column has NaN values, and groupby drops them by default. I pass `dropna=False` to keep them as their own group.

88. The same code works in your notebook but gives a different result on the full data. What do you check?

Whether I tested on a sample, whether new categories or missing values appeared in the full data, whether data types changed while reading, and whether any merge created duplicate rows.

PART 6

Master Cheat Sheet

Written as "I want to do this → write that". Last 20 minutes before the interview, read only this part.

▸ How to read this cheat sheet

Left column = **what you want to do, in plain words**. Right column = **the code to write**. Find your sentence on the left, copy the code on the right.

NumPy

I want to...	Write this
make an array from a list	`np.array([1,2,3])`
make an empty table of 0s / 1s	`np.zeros((2,3))  np.ones((2,3))`
make numbers 0 to 10 jumping by 2	`np.arange(0,10,2)`
make exactly 50 points between 0 and 1	`np.linspace(0,1,50)`
make random numbers (same every run)	`np.random.seed(42); np.random.rand(3)`
know rows and columns	`a.shape`
know the data type / change it	`a.dtype  a.astype(float)`
take one full column of a 2-D array	`m[:, 1]`
take rows 0–1 and columns 1–2	`m[0:2, 1:3]`
pick specific positions (0, 3, 5)	`a[[0,3,5]]`
keep only values bigger than 50	`a[a > 50]`
keep values between 20 and 80	`a[(a > 20) & (a < 80)]`
write if-else on the whole array	`np.where(a > 50, 'pass', 'fail')`
get a real copy, not a linked view	`b = a[1:4].copy()`
total / average of each column	`a.sum(axis=0)  a.mean(axis=0)`
average of each row	`a.mean(axis=1)`
biggest value / its position	`a.max()  a.argmax()`
sort the values	`np.sort(a)`
unique values and how many times each	`np.unique(a, return_counts=True)`
change shape to 3 rows (auto columns)	`a.reshape(3,-1)`
make it a single flat line	`a.ravel()  (copy: a.flatten())`
join two arrays / split one	`np.vstack([a,b])  np.split(a,3)`
multiply two matrices	`A @ B`
make every column mean 0, std 1	`(a - a.mean(axis=0)) / a.std(axis=0)`
find / ignore missing values	`np.isnan(a)  np.nanmean(a)`
get quartiles (25%, 50%, 75%)	`np.percentile(a, [25,50,75])`

Pandas

I want to...	Write this
read a CSV file	`df = pd.read_csv('f.csv')`
read only some columns	`pd.read_csv('f.csv', usecols=['a','b'])`
read a date column properly	`pd.read_csv('f.csv', parse_dates=['date'])`
see the first rows / a random sample	`df.head()` `df.sample(5)`
see types and missing counts together	`df.info()`
see mean, min, max of all numeric columns	`df.describe()`
count missing values in each column	`df.isnull().sum()`
take one column / many columns	`df['a']` `df[['a','b']]`
take rows by position (first 5)	`df.iloc[0:5]`
take rows by label / condition	`df.loc[df.age > 25, ['name','city']]`
filter on two conditions	`df[(df.age > 25) & (df.city == 'Delhi')]`
filter where value is in a list	`df[df.city.isin(['Delhi','Pune'])]`
filter where text contains something	`df[df.name.str.contains('an', na=False)]`
count how many of each category	`df['city'].value_counts()`
get the percentage share instead	`df['city'].value_counts(normalize=True)`
sort by salary, highest first	`df.sort_values('salary', ascending=False)`
get the top 3 rows by a column	`df.nlargest(3, 'salary')`
remove duplicate rows	`df.drop_duplicates(subset=['id'])`
average salary per department	`df.groupby('dept')['salary'].mean()`
many calculations per group	`df.groupby('dept').agg(avg=('salary','mean'), n=('salary','count'))`
put the group average on every row	`df.groupby('dept')['salary'].transform('mean')`
join two tables on a common column	`pd.merge(a, b, on='id', how='left')`
stack two tables one below the other	`pd.concat([a, b], ignore_index=True)`
make an Excel-style pivot table	`df.pivot_table(index='team', columns='year', values='runs', aggfunc='mean')`
count table of two categories	`pd.crosstab(df.team, df.result)`
check which columns relate to each other	`df.corr(numeric_only=True)`
rename a column	`df.rename(columns={'old':'new'})`
make a column the row label / undo it	`df.set_index('id')` `df.reset_index(drop=True)`
drop rows where salary is missing	`df.dropna(subset=['salary'])`
fill missing age with the median	`df['age'].fillna(df['age'].median())`
fill forward (time series)	`df.ffill()`
convert text to number / to date	`pd.to_numeric(df.a, errors='coerce')` `pd.to_datetime(df.d)`
save memory on a repeated text column	`df['city'] = df['city'].astype('category')`
make a new if-else column	`df['tag'] = np.where(df.age > 30, 'old', 'young')`
turn a number column into groups	`pd.cut(df.age, bins=[0,18,35,60], labels=['kid','young','old'])`
make 4 equal-sized groups	`pd.qcut(df.salary, 4)`

7-day moving average	`df['sales'].rolling(7).mean()`
month-wise total from daily data	`df.set_index('date').resample('M').sum()`
save the result to CSV	`df.to_csv('out.csv', index=False)`

Matplotlib

I want to...	Write this
start one chart	`fig, ax = plt.subplots(figsize=(8,4))`
start a 2×2 grid of charts	`fig, ax = plt.subplots(2, 2, figsize=(12,8))`
draw a trend line	`ax.plot(x, y, color='b', ls='--', marker='o', label='2024')`
draw dots for 2 numeric columns	`ax.scatter(x, y, c=z, s=sizes, alpha=.6)`
draw bars (vertical / horizontal)	`ax.bar(cat, val)  ax.barh(cat, val)`
see the distribution of one column	`ax.hist(values, bins=30, edgecolor='black')`
see outliers of one column	`ax.boxplot(values)`
add title and axis names	`ax.set_title('t'); ax.set_xlabel('x'); ax.set_ylabel('y')`
show the legend	`ax.legend(loc='best')  (needs label= in the plot)`
rotate crowded x labels	`plt.xticks(rotation=45)`
fix overlapping labels	`plt.tight_layout()`
one title for the whole figure	`fig.suptitle('Dashboard')`
point an arrow at a value	`ax.annotate('peak', xy=(5,25), xytext=(7,30), arrowprops=dict(arrowstyle='->'))`
draw a target line across the chart	`ax.axhline(50, color='red', ls='--')`
use a second y-axis (different unit)	`ax2 = ax.twinx()`
remove the top and right border	`ax.spines['top'].set_visible(False)`
change the whole look in one line	`plt.style.use('ggplot')`
set a default figure size for all charts	`plt.rcParams['figure.figsize'] = (10,5)`
save the chart (do this BEFORE show)	`plt.savefig('c.png', dpi=300, bbox_inches='tight')`

Seaborn

I want to...	Write this
set a nice theme once	`sns.set_theme(style='whitegrid')`
dots for 2 numeric columns, coloured by a category	`sns.scatterplot(data=df, x='bill', y='tip', hue='sex')`
trend line with a confidence band	`sns.lineplot(data=df, x='day', y='sales')`
remove that shaded band	`sns.lineplot(..., errorbar=None)`
count of each category	`sns.countplot(data=df, x='class')`
average per category with error bars	`sns.barplot(data=df, x='day', y='bill')`
compare spread across groups	`sns.boxplot(data=df, x='day', y='bill')`
same, but also show the shape	`sns.violinplot(data=df, x='day', y='bill', split=True, hue='sex')`
show every single data point	`sns.stripplot(..., jitter=.25)` or `swarmplot`
distribution of one numeric column	`sns.histplot(data=df, x='bill', bins=30, kde=True)`
smooth density curve only	`sns.kdeplot(data=df, x='bill', fill=True)`
correlation heatmap	`sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', vmin=-1, vmax=1)`
all columns against all columns	`sns.pairplot(df, hue='target', corner=True)`
scatter plus side distributions	`sns.jointplot(data=df, x='bill', y='tip', kind='hex')`
scatter with a regression line	`sns.regplot(data=df, x='bill', y='tip')`
one separate panel per category	`sns.catplot(data=df, x='day', y='bill', kind='box', col='sex')`
put a seaborn plot inside my own subplot	`sns.boxplot(..., ax=ax[0,0])` (axes-level only)
make a figure-level plot bigger	`sns.catplot(..., height=5, aspect=2)` (not figsize)

▸ 20 Golden Rules (remember these, not the syntax)

1	**axis = the direction that disappears.** axis=0 works down the rows (answer per column), axis=1 works across the columns (answer per row).
2	A NumPy **slice is a view** (shared memory); fancy and boolean indexing give copies. Use `.copy()` when in doubt.
3	In filters use `&` `|` `~` with brackets — never `and` / `or`.
4	`loc` = by name, end **included**. `iloc` = by position, end **excluded**.
5	To change values use `df.loc[mask, 'col'] = x`, never `df[mask]['col'] = x`.
6	Pandas returns a **new** object — assign it back: `df = df.dropna()`.
7	Broadcasting: compare shapes from the right; each pair must be equal or one must be 1.
8	`*` is element-wise, `@` is matrix multiplication.
9	`max` gives the value, `argmax` gives the position.
10	groupby = **split → apply → combine**. `agg` shrinks rows, `transform` keeps the same rows.
11	After a merge, **always check the row count**. More rows than before means a duplicated key.
12	Fill with **median** when outliers exist, mean only on clean data, mode for text.
13	`NaN == NaN` is False. Always use `isnull()` / `isnan()`.
14	Histogram = numeric ranges (bars touch). Bar chart = categories (bars have gaps).
15	Bar chart y-axis **must start at 0**; line charts need not.
16	`savefig()` comes **before** `show()`, otherwise the image is blank.
17	Seaborn: axes-level takes `ax=` and `figsize`; figure-level takes `col/row` and `height/aspect`.
18	Seaborn needs **long (tidy)** data for `hue` — one row = one observation. Use `melt()` if it is wide.
19	Prefer vectorized code over `apply`, and `apply` over `iterrows`.
20	Correlation is **never** proof of cause.

▸ Top 15 Mistakes (avoid these and you are already ahead)

1	Using `and`/`or` instead of `&`/`|`, or forgetting brackets in a filter.
2	Chained assignment `df[mask]['col'] = x` — use `df.loc[mask,'col'] = x`.
3	Forgetting that Pandas methods return a new object — `df.dropna()` alone changes nothing.
4	Mixing up loc (end included) and iloc (end excluded).
5	Forgetting `index=False` in `to_csv`.
6	Merging on a duplicated key and silently getting extra rows.
7	Filling missing values with the mean when outliers exist — use the median.
8	Doing maths on numbers stored as text, or comparing dates stored as text.
9	Using `apply`/`iterrows` where a vectorized operation exists.
10	Forgetting that a NumPy slice is a view, so the original changes too.
11	Checking NaN with `==` instead of `isnan`/`isnull`.
12	Calling `savefig()` after `show()` and getting a blank file.
13	Expecting `figsize` to work on a Seaborn figure-level plot.
14	Drawing a bar chart whose y-axis does not start at 0.
15	Saying a high correlation proves a cause.

FINAL TIP FOR THE INTERVIEW

Never answer with only a function name. Add **why** — "two numeric columns, so a scatter plot", "outliers present, so median not mean", "the key repeats, so I checked the shape after the merge". That one habit is the difference between someone who memorised the syntax and someone who has actually worked with data.